

1: Lots of tools and a simple program

Due by noon, Monday, Aug 28, 2023

In this homework, we primarily familiarize ourselves with a number of tools we'll be using throughout the semester. In order to get something running, and manage to finish the homework on time, you probably have to get it all right, so please get started early as there are many stumbling blocks along the way.

The instructions below are very minimal. Expect to have to do some googling along the way to figure out how to get things to work on your end. For a great manual of git, see <http://git-scm.com/book>. Make sure to read sections 1-3 to get a good understanding of its operation.

Get yourself a Linux machine

For this class, you need access to a Linux machine. It is convenient if your laptop runs a native Linux, but many other solutions will work as well. The recommended setup if you don't already have a machine running Ubuntu Linux is to use a virtual machine to run Ubuntu Linux.

VMWare, for which a free license will be provided on your platform of choice, offers an excellent virtual machine environment. (alternatives: virtualbox, parallel desktop, etc.). If you were registered for the class by 10 am on the first day of class, you should already have been provided a VMWare license. If you did not receive instructions in email, try this:

- Go to <http://go.uic.edu/csvmware>
- Click on the "sign in" link at the top
- click on "register"
- select "An account has been created..." and continue with the registration. Your account is your UIC email address.

If you were not registered by that time, send an email to Phil Beltran <pbeltr1@uic.edu> requesting a VMWare license for CS461, and cc: me <jakob@uic.edu>.

In principle, if you have a Mac, you don't need a Linux machine, but it will be easier, as some tools don't work quite the same, which can make things confusing. I would avoid WSL on Windows, but it could also work.

Install Ubuntu in a new virtual machine

From the VMWare menu, select "New". Download an Ubuntu install disk from here: <https://cdimage.ubuntu.com/jammy/daily-live/current/>. Choose the amd64.iso image if you have an x86 processor (most laptops), or the arm64.iso if you have an M1/M2 mac laptop. Once downloaded, drag it to the VMWare window as directed. A minimal desktop install should be fine. Then install some packages, using the terminal:

```
sudo apt update
sudo apt install build-essential
sudo apt install git
sudo apt install qemu-system-x86
```

if you have a mac, you'll also want an x86 cross compiler:

```
sudo apt install gcc-x86-64-linux-gnu
```

You can either use the virtual machine as a virtual desktop computer, and run everything for class inside of it. Or, you can use it as a virtual server, and connect to it from your normal desktop (work out how to do that - chatgpt can probably help, or ask the class). I prefer the server model myself. Whichever way you choose to go, make sure you have a good code editor. In 99% of cases, that is not vi, emacs, or nano. I recommend Visual Studio Code, with the Remote - SSH plugin.

Manual setup on Linux/MacOS/Windows (WITHOUT using a virtual machine)

It is possible, but not recommended, to do the course homeworks without the Ubuntu Linux virtual machine. This can be less resource intensive, and more convenient when it works, but it is more error prone and is not going to be supported in the class. That said, if you feel like experimenting, you could give it a shot.

On linux

Qemu is a PC emulator we'll be using to run xv6 under. In linux (Ubuntu),

```
sudo apt update
sudo apt install qemu-system-x86
```

should do it.

On Mac OS

install homebrew (<https://brew.sh/>), then

```
brew install git qemu x86_64-elf-gcc make
```

This works fine on both older macs and on M1/M2 macs.

On Windows (ask google for detailed instructions):

- Install WSL2.
- Install VcXsrv, and launch with multi-window, start-no-client, and grant all possible network permissions when prompted).
- Install a Linux distro (e.g., Debian or Ubuntu), and install necessary packages (gcc, qemu, etc.).
- Add the following two lines to ~/.bashrc

```
export DISPLAY=$(awk '/nameserver / {print $2; exit}' /etc/resolv.conf 2>/dev/null):0
export LIBGL_ALWAYS_INDIRECT=1
```

Other useful packages that might not come with your setup:

- build-essential (the compiler toolchain (Ubuntu Linux))
- git
- tig

Get the xv6 sources, build and run

Then fetch the version of the xv6 source tree from the public class repository:

```
git clone https://github.com/bitslab/xv6-public
```

Have a look around in this folder, and see how it compares to the booklet you received in class. Now, you are ready to build xv6, and run it in a Qemu VM: `$ make qemu`

study tip: Have a look at the Makefile, to see what the qemu target does. Follow its dependencies (fs.img, xv6.img), and see how they're built, and so on.

There's a lot to explore in this Makefile. If there's something you don't understand in it, try asking ChatGPT or another AI software for an explanation. Or if that doesn't help, ask on Piazza. Let's figure it out together.

in depth: have a look at the UPROGS variable. What's it for? How is it used? All those names, they show up in the qemu box when xv6 is up and running. How did they end up there? How did the leading underscore in the file names go away?

Create and checkout a new git branch for hw1

In the xv6 folder, create and check out a new branch based on the current HEAD: (-b: create a new branch)

```
git checkout -b hw1
```

Now you're in the hw1 branch and it is identical to the master branch.

study tip: how do you switch between branches? What's the point of a git branch anyway? What's the difference between git, and github? Do you know what a branch is?

in depth: branching and merging are really important concepts in git. Try making a small change to the master branch, after you created the hw1 branch. Then switch back to the hw1 branch, and merge in the changes from the master branch. You can ask chatgpt to show you how, but make sure you practice it yourself too to become proficient.

have a look at *git log*. What happens if you check out a commit somewhere in the middle? (Do you know how?) You get a big scary warning message, but it's not an error. Try to work out what a "detached HEAD" means, and why they warn about it. Ask an AI for clarification if you need it. If you delete a branch, what happens to the commits? Can you delete a commit?

Write a 'divide' program for xv6

xv6 comes with a number of small programs, like ls and cat. You are to create a new program, divide, **which takes two integer arguments on the command line**, and prints out the value of the first argument divided by the second argument.

Read the Makefile to figure out how to add a new program to the xv6 image.

Name the program divide.c, and make sure that typing

```
make qemu
```

or

```
make qemu-nox
```

automatically builds the divide binary just like it builds the other binaries. For a correct implementation, running divide in qemu should produce an output like (\$ represents the shell prompt):

```
$ divide 100 5
20
$ divide -4 5
-0.8
$ divide 2 3
0.66
```

As shown, output **up to two decimals, but no trailing zeroes**.

For **any** input, divide should produce either the correct number, or NaN if the input is invalid. You may expect integer inputs only.

Hint: xv6 does not provide a standard C library. Search ULIB in Makefile and see what are REALLY provided.

divide should not trigger any exceptions throughout its execution (you will see a line of "... pid ... trap ..." printed to the terminal when an exception is caught by the kernel). Initially, it probably will, as things work a little differently without libc. Can chatgpt work out why the program keeps crashing? I bet you can, or at least how to fix it.

study tip: the hard part here is working out how to write programs for xv6, not the actual program, though it is a bit awkward since we're used to having libc around. If you want, you can lean on AI help to write the program, but keep in mind: this is a pretty easy program to write yourself, and chatgpt could get confused by this weird context. If you can't work it out on your own, it means you need more practice, perhaps both with algorithmic thinking and with C programming. If you got help with the divide program, maybe think of a different small puzzle to work on, to hone your basic C programming skills.

in depth: there's a very simple printf, and there's no scanf at all. How would you write a small scanf?

Commit your work to the local repository

Stage your modified and added files

```
git add Makefile divide.c,
```

and commit your staged files

```
git commit -m "Homework 1, first attempt".
```

study tip: have a look at git log now, to see what you just did. There's a way to show all the contents of the commit too - try to work it out.

in depth: professional git users make tons of local commits, and work with multiple branches at the same time. And they generally don't keep lots of folders with different versions in them. That's for newbs who are scared of git. Try to get less scared of git. Try to get in the habit of committing whenever you've made some good progress, and leave a commit message that tells you what's in the commit, so you can come back to it when you screw things up later. Did you try git stash yet? If not, check it out. It's great.

Submit Homework Using Git

Create your hw1 turn-in repository by following this invitation link:

ATTN: [hw1 submission invitation](#)

Inside your xv6 folder, you now need to add your turn-in repository as a new *remote*. From github, copy the URL of the repository. Replace the "DUMMY" with the actual repo name. Then push your hw1 solution to your submission github repository:

```
git remote add turn-in http://github.com/CS461/hw1-DUMMY
git push turn-in hw1:hw1
```

alternative push methods:

If you used a different name for your local branch, or forgot to create a hw1 branc, you could also do:

```
git push turn-in anotherbranch:hw1
```

or

```
git push turn-in HEAD:hw1
```

to push whatever local branch currently being checked out.

Note: github has introduced a new authentication method, and retired the standard password method. When you try to push your work the first time, you will get an error message saying you need to get a "personal authentication token". This works just like another password, AFAIK. Just follow the instructions to get it.

study tip: make sure you get a decent grasp on what remote repositories are in git, and how to work with them: adding, removing, listing repositories, as well as remote branches. Why does git have local and remote repositories, instead of just one central one, like subversion/svn?

in depth: what's a tracking branch, and how is it different from a normal branch? Can you make a remote repo that isn't github hosted? try it! Can you make a local repository, that you share with a classmate?

Check your submission

To see that your submission was successfully pushed to your turn-in repository, try this in a new, empty directory:

- ```
git clone your personal github homework repo.
git checkout hw1
```

- see that the file you added and committed shows up.

```
make qemu
```

- run divide to make sure it is there and works.

**study tip:** when the qemu window pops up, that's not a regular command line terminal. That's a virtual machine running on your own OS: xv6. Try using the windows that are available on that terminal. Start with "ls". How is this terminal different from what you're used to?

**in depth:** the qemu window was started with a couple of files as arguments: xv6.img and fs.img. Neither of those is a command line shell. What program is it that is reading your commands, and running programs for you? Read the code for that program - hopefully you've seen something similar before (in CS361 or equivalent). If not, spend some time studying it, to see how a Unix command line shell works. Ask for help.

## Github ID

If your Github ID does not contain your UIC NetID. Please let us know!

<https://forms.gle/MZivfLPLbvD3VR2j8>

## Reminder: There will be a quiz on this homework!

A correct homework turn-in per the above instructions gives you only a small credit. Think of it as a participation prize. The Quiz, which will be in-class only on the day of the due date, is much more important in terms of class credit.

To do well on the quiz, you don't strictly need to finish the homework. But you do need to learn as much as you can from attempting the homework, and from getting maximum exposure to the material and topics directly related to the homework. The **study tips** and **in depth** suggestions should help.